

Instituto Politécnico Nacional

Escuela Superior de Cómputo

Web Client and Backend Development Frameworks.

Práctica: interfaz gráfica de escritorio para Carrera con JDBC y el patrón
DAO

Profesor: Dr. José Asunción Enríquez Zárate

Alumno: Angel Frausto Robles

Boleta: 2019630177

id634296@gmail.com

7CM2

19 de septiembre de 2026

Índice

1. Introducción	2
2. Desarrollo de la aplicación	2
2.1. La tabla en la base de datos	2
2.2. Las clases DAO y PreparedStatement	2
2.3. La interfaz gráfica y el CRUD	2
2.4. Evidencia de conexión JDBC y operaciones realizadas	8
3. Conclusiones	8
4. Referencias Bibliográficas	9

Índice de figuras

1.	Estructura real de la tabla Carrera (describe Carrera;)	2
2.	Estado inicial: solo Nuevo y Listar habilitados, tabla vacía	3
3.	Alta de la primera carrera, ya reflejada en la tabla tras Guardar	4
4.	Segunda alta, para tener dos registros y probar actualizar/eliminar	5
5.	Carrera 1 con la descripción actualizada vía JDBC	6
6.	Confirmación antes de eliminar la carrera 2	7
7.	Estado final: solo queda la carrera 1, ya actualizada	8
8.	Contenido real de la tabla tras las operaciones hechas desde la GUI	8

1. Introducción

Esta práctica conecta una aplicación de escritorio en Java con MySQL a través de JDBC, usando el patrón DAO para que la interfaz gráfica nunca escriba SQL directamente. El recurso es **Carrera**, con cuatro operaciones: alta, baja, cambio y consulta. La clase **CarreraDAO** concentra el acceso a datos con **PreparedStatement**, y la clase **VentanaCarrera** (Swing) dispara esos métodos según el botón que se presione.

2. Desarrollo de la aplicación

2.1. La tabla en la base de datos

La base **MisCursos** corre en un contenedor de MySQL 8 (**docker-compose.yml** de esta misma carpeta), con la tabla creada desde el script de inicialización. La figura 1 muestra la estructura real de la tabla, obtenida con `describe Carrera;` contra el contenedor en ejecución.

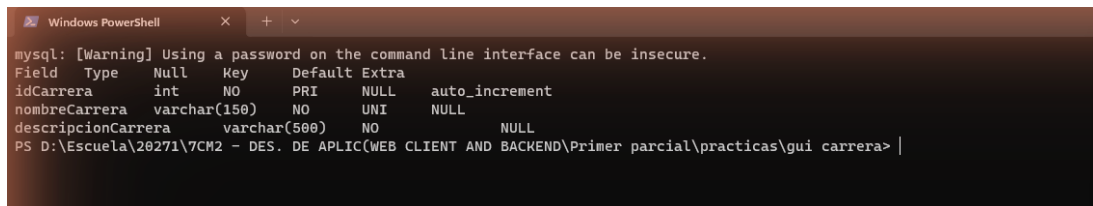


Figura 1: Estructura real de la tabla Carrera (`describe Carrera;`)

2.2. Las clases DAO y PreparedStatement

CarreraDAO tiene un método privado, `obtenerConexion()`, que carga el driver de MySQL y abre la conexión JDBC. Los cinco métodos públicos (`create`, `read`, `readAll`, `update`, `delete`) llaman primero a ese método y después arman una consulta con **PreparedStatement**, nunca concatenando el SQL con los valores a mano:

Código 1: Alta de una carrera con PreparedStatement

```
1 public void create(Carrera c) throws SQLException {
2     obtenerConexion();
3     PreparedStatement ps = conexion.prepareStatement(SQL_INSERT);
4     ps.setString(1, c.getNombreCarrera());
5     ps.setString(2, c.getDescripcionCarrera());
6     ps.executeUpdate();
7     ps.close();
8     conexion.close();
9 }
```

El **PreparedStatement** cumple dos funciones a la vez. Evita la inyección SQL, porque los valores se mandan por separado del texto de la consulta y no se pegan directamente en la cadena. Y de paso deja el texto del SQL fijo como constante (`SQL_INSERT`), en vez de reconstruirlo en cada llamada.

VentanaCarrera no importa ninguna clase de `java.sql`: solo conoce a **CarreraDAO** y a **Carrera**, y reacciona a las excepciones que el DAO declara (**SQLException**) mostrando un cuadro de diálogo. Esa es la separación que el patrón DAO impone entre la interfaz y el acceso a datos [1].

2.3. La interfaz gráfica y el CRUD

La ventana arranca con solo los botones **Nuevo** y **Listar** habilitados (figura 2). Al dar de alta una carrera se habilita el formulario y el botón **Guardar**; al guardar, **CarreraDAO.create()** inserta el registro real en MySQL y la tabla se refresca leyendo de vuelta con `readAll()` (figuras 3 y 4).

 Carrera

Carrera

id Carrera

nombre

descripcion

Nuevo

Guardar

Actualizar

Eliminar

Listar

Cancelar

ID	Nombre	Descripcion
----	--------	-------------

Figura 2: Estado inicial: solo Nuevo y Listar habilitados, tabla vacía

 Carrera

Carrera

id Carrera

nombre

descripcion

Nuevo

Guardar

Actualizar

Eliminar

Listar

Cancelar

ID	Nombre	Descripcion
1	Ingenieria en Sistemas Com...	Formacion en desarrollo de ...

Figura 3: Alta de la primera carrera, ya reflejada en la tabla tras Guardar

The screenshot shows a window titled 'Carrera' with a standard Windows interface (minimize, maximize, close buttons). On the right side, there is a vertical stack of buttons: 'Nuevo' (highlighted in blue), 'Guardar', 'Actualizar', 'Eliminar', 'Listar' (highlighted in blue), and 'Cancelar'. On the left side, there is a form with the title 'Carrera' and three input fields labeled 'id Carrera', 'nombre', and 'descripcion'. Below the form is a table with three columns: 'ID', 'Nombre', and 'Descripcion'. The table contains two rows of data. The first row has ID '1', 'Ingenieria en Sistemas Com...', and 'Formacion en desarrollo de ...'. The second row has ID '2', 'Ingenieria en Sistemas Elect...', and 'Formacion en electronica dig...'. Below the table is a large empty rectangular area.

ID	Nombre	Descripcion
1	Ingenieria en Sistemas Com...	Formacion en desarrollo de ...
2	Ingenieria en Sistemas Elect...	Formacion en electronica dig...

Figura 4: Segunda alta, para tener dos registros y probar actualizar/eliminar

Al seleccionar una fila de la tabla se habilitan **Actualizar** y **Eliminar**, y el formulario se llena con los datos de esa fila (el id queda de solo lectura). La figura 5 muestra la carrera 1 después de modificar su descripción y presionar **Actualizar**: el cambio ya viene de una consulta UPDATE real contra la base.

Carrera

id Carrera: 1

nombre: Ingenieria en Sistemas Computacionales

descripcion: 1 desarrollo de software, redes, bases de datos y sistemas.

Buttons: Nuevo, Guardar, Actualizar, Eliminar, Listar, Cancelar

ID	Nombre	Descripcion
1	Ingenieria en Sistemas Com...	Formacion en desarrollo de ...
2	Ingenieria en Sistemas Elect...	Formacion en electronica dig...

Figura 5: Carrera 1 con la descripción actualizada vía JDBC

Eliminar pide confirmación antes de ejecutar el DELETE (figura 6), y una vez confirmado la fila desaparece de la tabla porque vuelve a leerse de la base (figura 7).

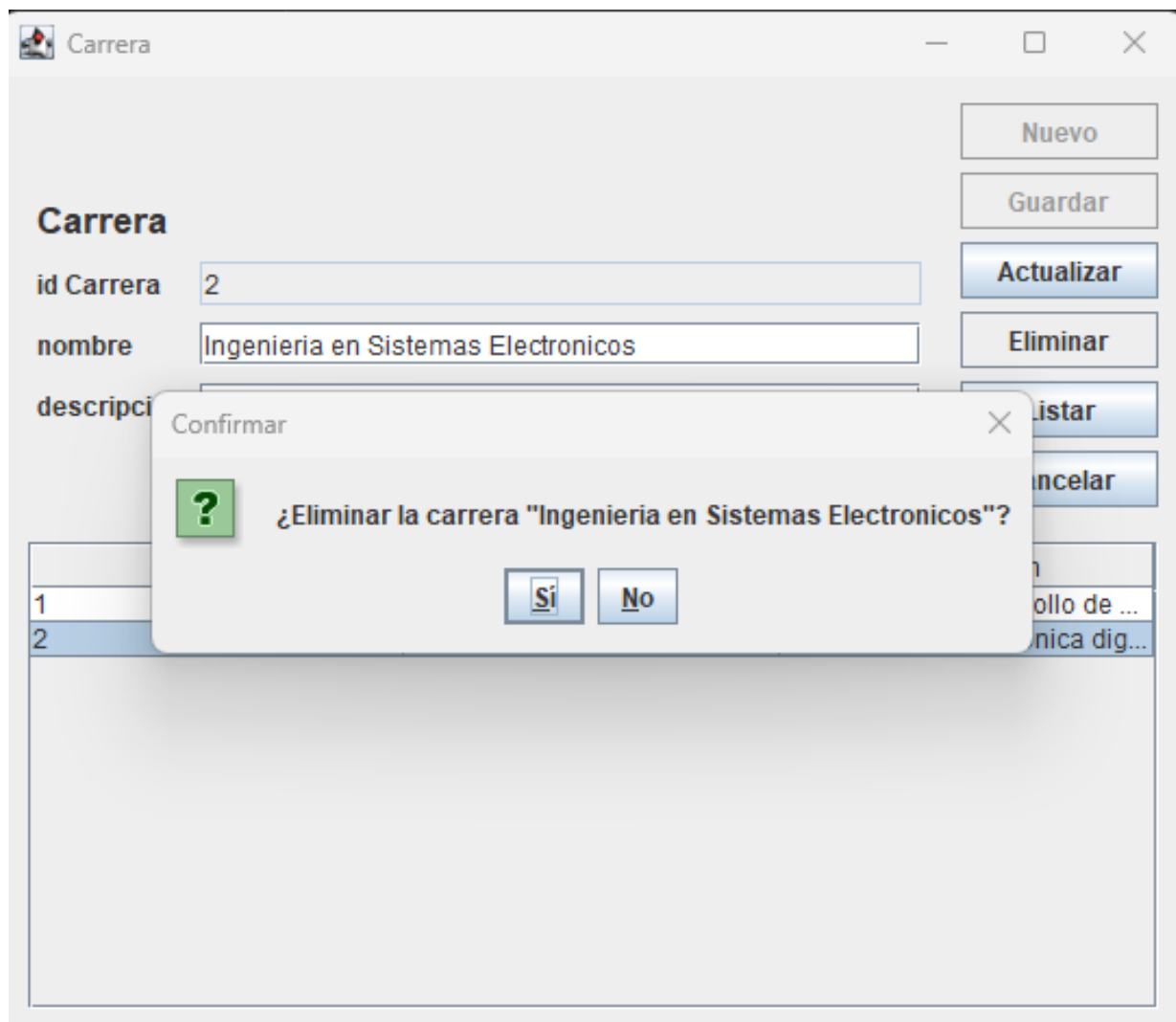


Figura 6: Confirmación antes de eliminar la carrera 2

Carrera

id Carrera: 1

nombre: Ingenieria en Sistemas Computacionales

descripcion: 1 desarrollo de software, redes, bases de datos y sistemas.

Buttons: Nuevo, Guardar, Actualizar, Eliminar, Listar, Cancelar

ID	Nombre	Descripcion
1	Ingenieria en Sistemas Com...	Formacion en desarrollo de ...

Figura 7: Estado final: solo queda la carrera 1, ya actualizada

2.4. Evidencia de conexión JDBC y operaciones realizadas

Para confirmar que la interfaz gráfica en verdad llega hasta la base (y no solo actualiza una lista en memoria), la figura 8 corre `select * from Carrera;` directo contra el contenedor de MySQL, después de terminar la secuencia de alta, actualización y baja hecha desde la ventana. El único registro que queda y su descripción coinciden exactamente con lo que muestra la figura 7.

```

mysql: [Warning] Using a password on the command line interface can be insecure.
idCarrera  nombreCarrera  descripcionCarrera
1          Ingenieria en Sistemas Computacionales  Formacion en desarrollo de software, redes, bases de datos y sistemas.
PS D:\Escuela\20271\7CM2 - DES. DE APLIC(WEB CLIENT AND BACKEND\Primer parcial\practicas\gui carrera>

```

Figura 8: Contenido real de la tabla tras las operaciones hechas desde la GUI

3. Conclusiones

Todas las excepciones que puede lanzar `CarreraDAO` caen bajo `SQLException`: falla de conexión, violación de la restricción `UNIQUE` en `nombreCarrera` al repetir un nombre, o un id que ya no existe al intentar actualizar o eliminar. `ClassNotFoundException` solo aparece si el conector de MySQL no está en el classpath, y por eso `obtenerConexion()` la atrapa y la convierte en `SQLException` para que el resto del DAO declare un único tipo de error.

El error más común al conectar por JDBC no es de código sino de configuración: la URL, el usuario o la contraseña no coinciden con el contenedor que en verdad está corriendo. Con el contenedor apagado o el puerto equivocado, la excepción llega igual hasta la GUI, pero como un mensaje de conexión rechazada en vez de un problema de SQL. Repetir un nombre de carrera es el segundo error típico, y es justo el que la restricción **UNIQUE** de la tabla convierte en una excepción controlada en vez de en un registro duplicado silencioso.

Separar **CarreraDAO** de **VentanaCarrera** tuvo un efecto concreto en esta práctica: cambiar la descripción de una carrera, agregar una fila o quitarla nunca tocó una línea de la ventana, porque toda esa lógica vive en el DAO. Esa es la relación entre el patrón DAO y la separación de responsabilidades que pedía el objetivo de la práctica, y coincide con lo que documenta el catálogo Core J2EE Patterns sobre este patrón [1].

4. Referencias Bibliográficas

- [1] Deepak Alur, John Crupi y Dan Malks. *Core J2EE Patterns: Best Practices and Design Strategies*. 2.^a ed. Prentice Hall, 2003.
- [2] Oracle Corporation. *Lesson: JDBC Introduction (The Java Tutorials)*. URL: <https://docs.oracle.com/javase/tutorial/jdbc/overview/index.html> (visitado 19-09-2026).
- [3] Oracle Corporation. *MySQL Connector/J Developer Guide: Overview of MySQL Connector/J*. URL: <https://dev.mysql.com/doc/connector-j/en/connector-j-overview.html> (visitado 19-09-2026).